Razorpay Seasons: AI build log

Track 2: Recover and Grow with AI · Prototype: razorpay-seasons.vercel.app · Code: this repository

I drove; AI was the engine. AI did the transcribing, searching, coding and checking. The reasoning, the problem and every call about what the merchant needs were mine.

1. AI tools and models used

Tool	Model	What I used it for
Claude (Anthropic), agentic workspace connected to GitHub and Vercel	Claude Opus 5.5	Research on Razorpay's AI launches (from launch-video transcripts I shared) and the brief; interview questions; building the whole prototype; deployment; automated browser testing; the Hindi dashboard translation; drafting the product note and this log from my notes
ChatGPT (OpenAI)	ChatGPT, default model	Product discovery and desk research: debating features, finding merchant discussions on Reddit and the open web, asking "what are we missing?"
Wispr Flow	Wispr Flow speech-to-text	Transcribing my merchant interviews, including the Hindi words merchants mixed in
Gemini (Google), *inside the product*	`gemini-3.1-flash-lite`, falling back to `gemini-2.5-flash-lite` / other current Flash models	Runs live in the prototype: translates customer messages into Hindi or Hinglish, makes them shorter or friendlier, and turns a merchant's season notes into changes for the next plan

Not AI, but used: Razorpay Payment Links API (test mode only), Razorpay Blade design system, GitHub, Vercel, Playwright (automated testing).

2. Key prompts, workflows and agent setup

Agent setup

- **Claude** ran as an agent in a cloud workspace with access to my GitHub repo and Vercel. It wrote code, ran builds, pushed to GitHub (Vercel auto-deploys from `main`), and checked every change in a headless browser with Playwright. That covered screenshots, a 15-step guided-tour test, and a crawl of every screen in Hindi to find untranslated text.
- I signed in to GitHub, Vercel, Google AI Studio and Razorpay myself and added the API keys to Vercel's environment variables myself. The AI never saw or typed a key.
- **Gemini** is called only from a Vercel serverless function (`api/ai.ts`). The key stays on the server and never reaches the browser.
- **Razorpay** is called from `api/razorpay.ts` in **test mode only**. Live keys are refused in code, and Razorpay's own SMS and email notifications are switched off, so nothing is sent to anyone.

Workflow

1. **Discovery:** I interviewed nine merchants (transcribed with Wispr Flow), studied one merchant's real dashboard, and researched Razorpay's AI launches and other entries in Razorpay's AI challenges.
2. **Reasoning:** I debated the problem and features with ChatGPT and Claude, asking both to be critical.
3. **First version:** I gave Claude a free hand on v1 to see a direction.
4. **Build-review iterations:** I reviewed each version and redirected it. Claude built, tested and deployed. I reviewed the live site and decided the next change. 22+ deployed versions.
5. **Write-up:** I gave my reasoning in long voice-typed paragraphs; Claude turned them into the product note and this log, and I edited the result.

Key prompts (my own words, typos fixed, otherwise as I said them)

These are the long prompts where I set direction. Most were dictated, which is why they read like speech.

Pushing back on the obvious problem

> "You know the problem is all of them have used AI, and AI has suggested them these. That is why we are trying out of the box. This cannot be the only problem of Razorpay."

Asking for criticism, not agreement

> "Find out a bit... research about it a bit more and let me know how much this idea is feasible. Be a critic here."

Grounding the idea in data before building

> "The seasonal thing is good... what all are we going to do in it? Suggest something on that layer, and also go through some real data first to understand how much of an issue it is."

Setting the non-negotiable rule

> "Anything that requires an interaction with customers needs an approval. That is the baseline. We are not moving on that. At the end of the day, it should first suggest, and then when it's approved, it ends up doing the task. We keep individual customer nudges as well... let's show all of them."

Making it look like Razorpay

> "This is the HTML and the CSS, to help you figure out how our agent should look like." *(Razorpay dashboard markup I shared, so the prototype uses Razorpay's own Blade design system.)*

Reviewing a version

> "Be critical of it first of all." Then: "Go ahead, one thing: do not make it crowded, and also update the tour accordingly, so that the judge can find out things easily."

Questioning a feature on ethics

> "Predictions from similar merchants: is not this privacy invasion?" *(This led to benchmarks that use only aggregated ranges from 50+ merchants and never show names.)*

Checking against the brief before writing

> "Before we move to the one-page product note, go through the brief and go through what a successful PM does. Find out things for that and then figure out if we are missing anything. If everything is fine, only then we move to the one-page product note."

Stopping a feature before it went in

> "Is it a good idea to add the product thinking part for the judges? I'm still not sure. Is it said somewhere? Because at the end of the day, we are building a prototype. So let's first figure that out if it's a good idea or not."

Making the write-up mine

> "These are my thoughts. Collate it and make it yours... The most important thing is that it should come from a person like me. Refine them and show them that this is why we made it."

---

3. AI output I rejected, corrected or significantly improved, and why

Gemini's season lessons invented numbers and suggested removing approval. The first version of "Review & learn" asked Gemini to turn a merchant's notes (for example, "we ran out of stock by day 4") into changes for next season. Its early answers:

- **Invented a figure** that wasn't in the data (for example, "increase stock by 25%").
- **Proposed automating** customer messages, which breaks the approval rule.
- **Misread the cause:** it lowered next year's demand forecast when sales had dropped only because stock ran out. That's a supply problem, not a demand one.

Why that mattered: a merchant acting on an invented number, or a system that quietly drops approval, is exactly the trust failure this product must not have.

How I fixed it:

1. I rewrote the prompt with explicit rules: use only numbers present in the data, never suggest automation or skipping approval, separate supply problems from demand, and flag one-off events.

2. I added **server-side checks** in `api/ai.ts` that drop any lesson containing a number or ₹ amount not found in the merchant's notes or data, and any lesson mentioning automation or skipping approval.
3. I added **client-side checks** on every Gemini message rewrite. A rewrite is rejected, and the merchant's text kept, if it drops the customer's name or the payment link, removes the STOP opt-out, adds a discount, changes an amount, or goes over the merchant's discount limit.

Also rejected from Claude:

- A "Product thinking" tab inside the prototype. I parked it, because the brief says the prototype should make the user journey tangible and the reasoning belongs in the product note.
- A multi-agent setup per business type, which ChatGPT and I explored early. I dropped it as too broad for the merchant's actual problem.

---

4. What AI did vs what I decided or built myself

AI did	I decided
Transcribed interviews (Wispr Flow) and summarised them (Claude)	**The problem:** going back over the transcripts, I saw that every merchant faced seasonality and assumed they could handle it alone. No one, including me, named it in the room.
Researched Razorpay's AI launches and the open web (Claude, ChatGPT)	**The angle:** preparing merchants before the moment instead of recovering failed payments, which is where AI tools were steering everyone.
Wrote and deployed all the code (Claude)	**Approval is the baseline:** nothing reaches a customer or moves money without the merchant.
Built screens on Razorpay's Blade design system (Claude)	**Information architecture:** progressive disclosure, a tree-shaped journey that starts from one tile and opens outward, because merchants didn't know features Razorpay already had.
Drafted UI copy and the Hindi translation (Claude)	**What the screens show:** the headline tile, the "Am I ready?" task bars, the Quick insights carousel, the editable WhatsApp message, Hindi, the WhatsApp cost before approval.
Runs translation, rewrites and lessons live in the product (Gemini)	**Scope:** three layers (year, month, week) and predictive mode for new merchants. Left out: a chatbot, auto-sending, new payment products, per-business agents, and a product-thinking tab.
Turned my spoken reasoning into the note and this log (Claude)	**Metrics:** adoption as the north star (merchants acting on their plan each season), revenue against a matched holdout group, and opt-outs as a guardrail.

---

5. Known limitations, risks and verification steps

Limitations

- **Synthetic data.** All merchants, customers and numbers in the prototype are made up, shaped on my interviews and one merchant's dashboard. Names are fictional. "Similar merchant" benchmarks are illustrative.
- **Simulated date.** The prototype is fixed at Mon 7 Sep 2026 so the Diwali flow can be shown.
- **Test mode only.** Payment Links are real Razorpay test-mode links. Nothing is sent to customers and no money moves. Only the first 5 links per approval are created, to stay within test-mode rate limits.
- **WhatsApp is simulated.** The WhatsApp cost uses clearly labelled example rates (₹1 marketing, ₹0.15 utility per message). Real rates vary by provider.
- **Some suggested features aren't confirmed in Razorpay today.** The app labels them "Suggested · not found in Razorpay's dashboard or docs", for example UPI retry after a card failure, and time-slot booking.
- **Hindi dashboard.** I translated it myself with AI help. A fluent reviewer hasn't checked it yet. Names, brands and amounts stay in English on purpose.

Risks

- **AI hallucination:** an invented number or a wrong cause in a merchant's plan. *Mitigation:* evidence and confidence shown on every suggestion, the server- and client-side checks above, and the merchant approves everything.
- **Customer annoyance or spam:** *Mitigation:* opted-in customers only, a STOP line in every message, quiet hours, a monthly message cap, and a discount limit. Loosening any of these asks the merchant to confirm.

- **Privacy of peer benchmarks:** *Mitigation:* only aggregated ranges from 50+ similar merchants, never individual data or names.
- **Over-reliance:** a merchant approving without reading. *Mitigation:* the approval panel shows who gets it, the exact message, and the cost before the button.

Verification steps I ran

- Live Gemini calls tested for Hindi and Hinglish translation (about 2–6 seconds each) and for lessons. Checked that the rules reject bad outputs.
- Razorpay test links created on approval and confirmed in the Razorpay test dashboard. Live keys refused.
- An automated Playwright run of the 15-step tour after every change: every step highlights the right element with no errors.
- An automated crawl of every tab for all four demo merchants in Hindi, to find untranslated text. Screenshots checked for layout breaks.
- The product note checked against the brief's Track 2 questions and submission requirements.

What I would test next

The core hypothesis, that preparing beats recovering, in a 3-month pilot with about 20 real merchants against a holdout group. The measures would be adoption, peak-window revenue, on-time collection, and opt-outs.

No API keys, credentials or private merchant data are included in this file or this repository. Keys live only in Vercel environment variables.